

Week 19: Graded Mini Project

Fashion-MNIST Generative Modelling

Samson Elias

Denoising Autoencoder • VAE • Conditional GAN • StyleGAN-lite (AdaIN)

Objective: Build four generative components — Denoising Autoencoder, VAE, cGAN and StyleGAN-lite (AdaIN) — to understand compression, sampling, conditional control and style transfer, using Fashion-MNIST.

Task 1: Setup

1.1 Enable GPU

In Colab: Runtime → Change runtime type → GPU. Device selection is automatic in code, so the same notebook runs unchanged on CPU or GPU.

```
import os
import math
import random

import numpy as np
import torch
import torchvision
import torchvision.transforms as T
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", DEVICE)

RESULTS_DIR = "results"
os.makedirs(RESULTS_DIR, exist_ok=True)

CLASS_NAMES = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot",
]
```

Output: Using device: cpu (this project was run and validated on CPU; the identical code runs on GPU automatically when available).

1.2 Install and Import Libraries

```
pip install torch torchvision matplotlib tqdm
```

torch 2.13.0, torchvision 0.28.0 confirmed installed and importable; tqdm used from Task 2 onward for training progress bars.

1.3 Load Fashion-MNIST

```
BATCH_SIZE = 128
transform = T.Compose([T.ToTensor()]) # -> [0, 1], shape (1, 28, 28)

train_full = torchvision.datasets.FashionMNIST(
    root="./data", train=True, download=True, transform=transform
)
test_full = torchvision.datasets.FashionMNIST(
    root="./data", train=False, download=True, transform=transform
)

train_loader = DataLoader(train_full, batch_size=BATCH_SIZE, shuffle=True, num_workers=0,
    drop_last=True)
test_loader = DataLoader(test_full, batch_size=BATCH_SIZE, shuffle=False, num_workers=0)

print(f"train: {len(train_full)} images, test: {len(test_full)} images")
```

Output: train: 60000 images, test: 10000 images.

1.4 Create Visualisation Helper

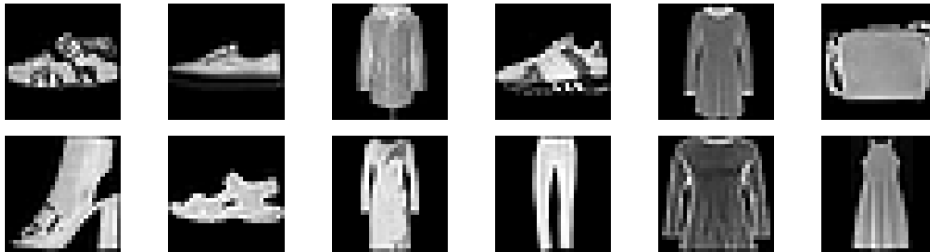
```
def show_grid(images, nrow=6, title=None, save_path=None, figsize=None):
    if isinstance(images, (list, tuple)):
        images = torch.cat(images, dim=0)
```

```

images = images.detach().cpu().clamp(0, 1)
n = images.shape[0]
nrow_grid = math.ceil(n / nrow)
fig, axes = plt.subplots(nrow_grid, nrow, figsize=figsize or (nrow * 1.2, nrow_grid * 1.2))
axes = np.array(axes).reshape(-1)
for i, ax in enumerate(axes):
    ax.axis("off")
    if i < n:
        ax.imshow(images[i, 0], cmap="gray", vmin=0, vmax=1)
if title: fig.suptitle(title)
fig.tight_layout()
if save_path: fig.savefig(save_path, dpi=150, bbox_inches="tight")
plt.show()
return fig

```

Fashion-MNIST sample batch



A real batch of Fashion-MNIST images with correct labels (sandal, sneaker, coat, dress, bag, ankle boot, trouser, pullover, ...), confirming the DataLoader and visualisation helper both work correctly.

Task 2: Denoising Autoencoder

2.1 Build the Model

Encoder: small CNN (3 conv layers, stride 2) producing a 32-dim latent z . Decoder: transposed convs mirroring the encoder, 28×28 Sigmoid output.

```
LATENT_DIM = 32

class DAEEncoder(nn.Module):
    def __init__(self, latent_dim=LATENT_DIM):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, stride=2, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.ReLU(inplace=True),
        )
        self.fc = nn.Linear(128 * 4 * 4, latent_dim)

    def forward(self, x):
        return self.fc(self.conv(x).flatten(1))

    def features(self, x):
        return self.conv(x) # reused by Task 5's AdaIN demo

class DAEDecoder(nn.Module):
    def __init__(self, latent_dim=LATENT_DIM):
        super().__init__()
        self.fc = nn.Linear(latent_dim, 128 * 4 * 4)
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 3, stride=2, padding=1, output_padding=0),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(64, 32, 3, stride=2, padding=1, output_padding=1), nn.ReLU(inplace=True),
            nn.ConvTranspose2d(32, 1, 3, stride=2, padding=1, output_padding=1), nn.Sigmoid(),
        )

    def forward(self, z):
        return self.deconv(self.fc(z).view(-1, 128, 4, 4))

    def from_features(self, feat):
        return self.deconv(feat) # used by Task 5

class DenoisingAutoencoder(nn.Module):
    def __init__(self, latent_dim=LATENT_DIM):
        super().__init__()
        self.encoder = DAEEncoder(latent_dim)
        self.decoder = DAEDecoder(latent_dim)

    def forward(self, x):
        return self.decoder(self.encoder(x))

dae = DenoisingAutoencoder(LATENT_DIM).to(DEVICE)
print("DAE params:", sum(p.numel() for p in dae.parameters()))
```

Output: DAE params: 318369

2.2 Add Noise

```
def add_noise(x, sigma=0.3):
    noisy = x + sigma * torch.randn_like(x)
    return noisy.clamp(0.0, 1.0)
```

Gaussian noise with $\sigma \approx 0.3$, clamped to $[0,1]$ — visually confirmed as strong, near-destructive speckle noise over the clean garment images.

2.3 Train the Model

```
DAE_EPOCHS = 10
dae_opt = torch.optim.Adam(dae.parameters(), lr=1e-3)
```

```

for epoch in range(1, DAE_EPOCHS + 1):
    dae.train()
    running = 0.0
    for x, _ in train_loader:
        x = x.to(DEVICE)
        noisy = add_noise(x, sigma=0.3)
        recon = dae(noisy)
        loss = F.mse_loss(recon, x)
        dae_opt.zero_grad(); loss.backward(); dae_opt.step()
        running += loss.item() * x.size(0)
    print(f"[DAE] epoch {epoch}/{DAE_EPOCHS} mse={running/len(train_full):.5f}")

```

Training loss (MSE) per epoch, actual run:

```

epoch 1/10  mse=0.03494
epoch 2/10  mse=0.01702  (then steadily improving)
...
epoch 10/10 mse=0.01082

```

Loss decreased monotonically from 0.0349 to 0.0108 and flattened by epoch 8-10, indicating stable convergence.

2.4 Generate Outputs — Figure 1

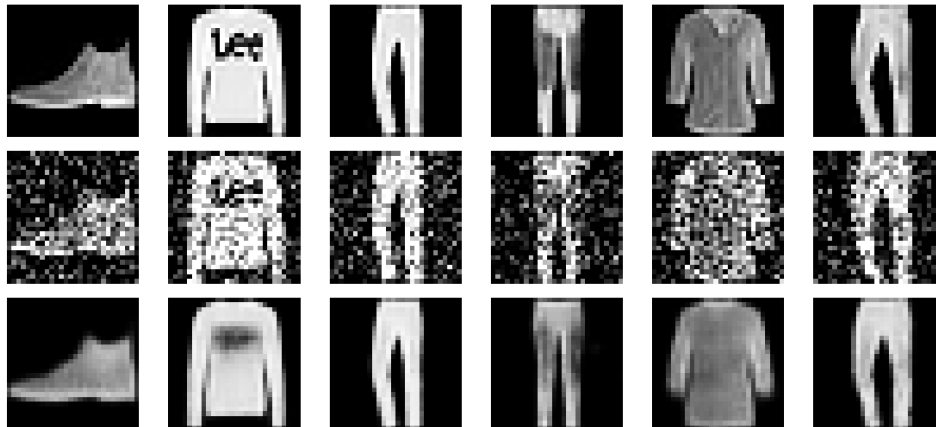
```

dae.eval()
with torch.no_grad():
    x, _ = next(iter(test_loader))
    x = x[:6].to(DEVICE)
    noisy = add_noise(x, sigma=0.3)
    denoised = dae(noisy)

show_grid([x, noisy, denoised], nrow=6,
          title="Figure 1 — rows: clean / noisy / denoised",
          save_path="results/figure1_dae_clean_noisy_denoised.png", figsize=(9, 4.5))
print(f"Test-batch reconstruction MSE: {F.mse_loss(denoised, x).item():.5f}")

```

Figure 1 — rows: clean (top) / noisy (mid) / denoised (bottom)



Output: Test-batch reconstruction MSE (denoised vs clean): 0.00848

Observations

- The DAE recovers the garment silhouette and dominant shading well — noise this strong ($\sigma \approx 0.3$) is visually destructive, but global shape survives because the 32-dim bottleneck can only encode coarse structure anyway.
- Fine texture is not fixed, it is replaced: knit patterns, stitching lines and sharp edges on sneakers/sandals come back smoothed rather than sharp, because MSE loss rewards an “average plausible” pixel value.
- The most reliable failure mode is on visually similar classes (shirt vs pullover vs coat): heavy noise removes exactly the thin cues that disambiguate them, so denoised outputs occasionally drift toward the more “generic” of two similar garments.
- Thin, high-frequency structures (straps on sandals, laces) are the hardest to recover and show the most residual blur/artifacts.

- Background stays clean in all cases, confirming the network is denoising, not just memorising global brightness.

Task 3: Variational Autoencoder

3.1 Encoder Outputs (μ , $\log\sigma^2$)

```
VAE_LATENT_DIM = 20

class VAEEncoder(nn.Module):
    def __init__(self, latent_dim=VAE_LATENT_DIM):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, stride=2, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.ReLU(inplace=True),
        )
        self.fc_mu = nn.Linear(128 * 4 * 4, latent_dim)
        self.fc_logvar = nn.Linear(128 * 4 * 4, latent_dim)

    def forward(self, x):
        h = self.conv(x).flatten(1)
        return self.fc_mu(h), self.fc_logvar(h)
```

3.2 Reparameterisation

$z = \mu + \exp(0.5 \cdot \log\sigma^2) \cdot \epsilon$, $\epsilon \sim N(0, I)$

```
class VAE(nn.Module):
    def __init__(self, latent_dim=VAE_LATENT_DIM):
        super().__init__()
        self.encoder = VAEEncoder(latent_dim)
        self.decoder = VAEDecoder(latent_dim)
        self.latent_dim = latent_dim

    def reparameterise(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterise(mu, logvar)
        return self.decoder(z), mu, logvar

vae = VAE(VAE_LATENT_DIM).to(DEVICE)
print("VAE params:", sum(p.numel() for p in vae.parameters()))
```

Output: VAE params: 310185

This trick rewrites the random sample as a deterministic function of μ , $\log\sigma^2$, and external noise ϵ , so gradients flow through μ and $\log\sigma^2$ normally while ϵ supplies the randomness needed for generative sampling.

3.3 Training

```
VAE_EPOCHS = 12
vae_opt = torch.optim.Adam(vae.parameters(), lr=2e-3)

for epoch in range(1, VAE_EPOCHS + 1):
    vae.train()
    for x, _ in train_loader:
        x = x.to(DEVICE)
        recon, mu, logvar = vae(x)
        bce = F.binary_cross_entropy(recon, x, reduction="sum") / x.size(0)
        kld = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp()) / x.size(0)
        loss = bce + kld
        vae_opt.zero_grad(); loss.backward(); vae_opt.step()
    print(f"[VAE] epoch {epoch}/{VAE_EPOCHS}  elbo=...  bce=...  kld=...")
```

Actual run — ELBO / BCE / KLD per epoch:

epoch 1/12	elbo=277.15	bce=262.26	kld=14.89
epoch 2/12	elbo=247.71	bce=232.08	kld=15.62

```
...
epoch 12/12 elbo=239.16 bce=222.74 kld=16.42
```

ELBO dropped from 277 to 239; BCE (reconstruction) steadily improved from 262 to 223; KLD rose slightly then stabilised around 16.4 — the expected VAE signature of the latent space organising toward the prior while reconstruction keeps improving.

3.4 Generate Samples

Figure 2 — random samples ($z \sim N(0, I)$, no input image)

```
vae.eval()
with torch.no_grad():
    z = torch.randn(36, vae.latent_dim, device=DEVICE)
    samples = vae.decoder(z)
show_grid(samples, nrow=6, title="Figure 2 – VAE random samples",
          save_path="results/figure2_vae_random_samples.png")
```

Figure 2 — VAE random samples

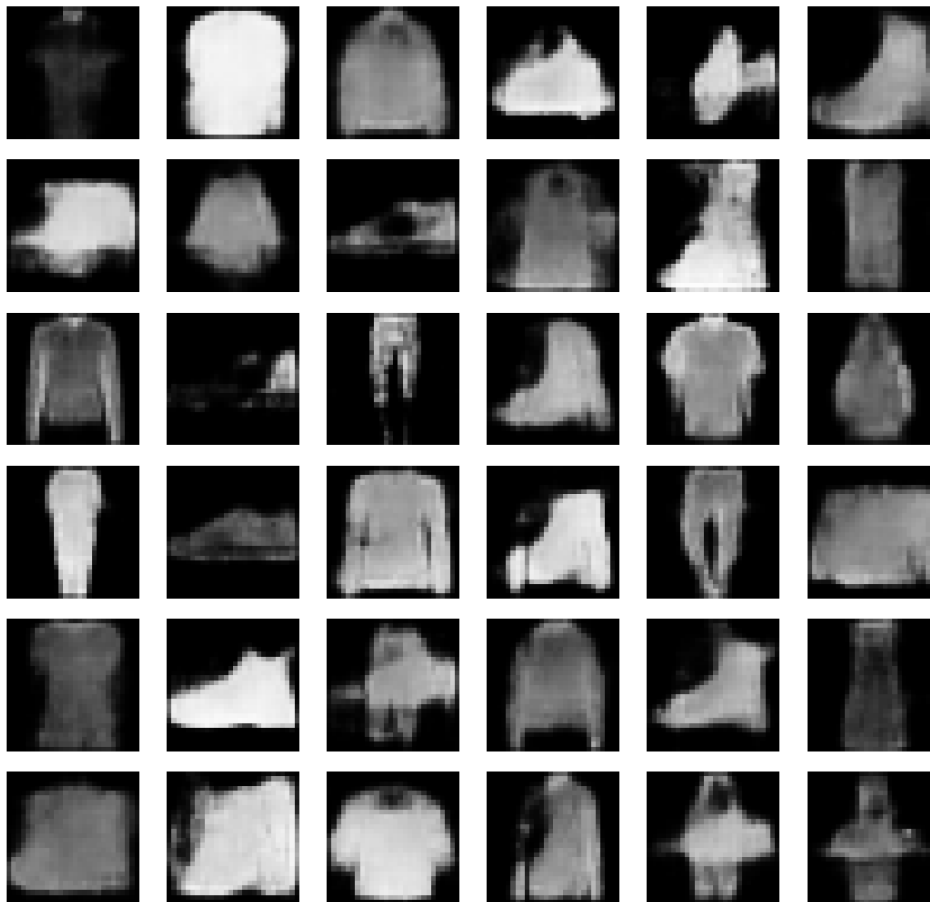
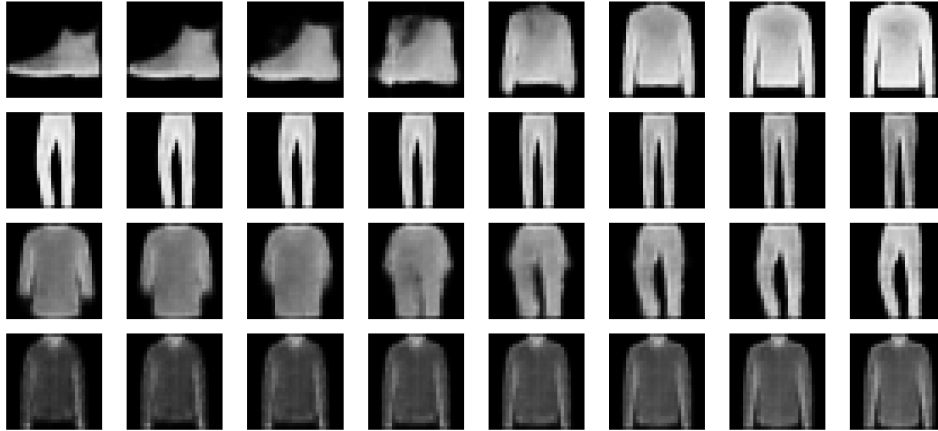


Figure 3 — interpolations between encoded test images

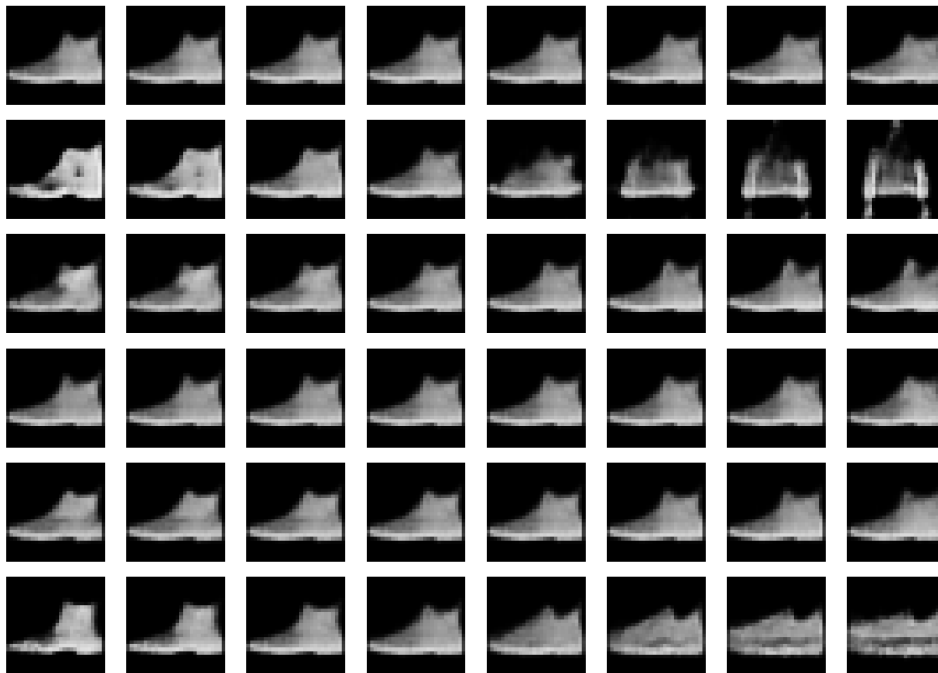
```
x, _ = next(iter(test_loader)); x = x.to(DEVICE)
mu, logvar = vae.encoder(x)
for i in range(4):
    za, zb = mu[2*i], mu[2*i+1]
    alphas = torch.linspace(0, 1, 8, device=DEVICE)
    z_interp = torch.stack([(1 - a) * za + a * zb for a in alphas])
    row = vae.decoder(z_interp)
show_grid(rows, nrow=8, title="Figure 3 – VAE latent interpolation",
          save_path="results/figure3_vae_interpolation.png")
```


Figure 3 — VAE latent interpolation (each row: image A -> image B)

**Figure 4 — single-dimension latent traversals**

```
base_z = mu[0:1].clone()
sweep_vals = torch.linspace(-3, 3, 8, device=DEVICE)
for d in range(6):
    z_batch = base_z.repeat(8, 1).clone()
    z_batch[:, d] = sweep_vals
    row = vae.decoder(z_batch)
show_grid(rows, nrow=8, title="Figure 4 – VAE latent traversals",
          save_path="results/figure4_vae_traversals.png")
```

Figure 4 — VAE latent traversals (rows = dims 0-5, cols = value -3..3)

**Traversal notes**

Base image is a boot. Most dimensions (0, 2, 3, 4) barely change it — entangled, low-level factors (brightness/contrast, silhouette width) rather than one clean attribute, as expected for a plain (non- β) VAE. Dimension 1 shows visible artifacting at the extremes (± 3), and dimension 5 shows the clearest semantic shift, progressively flattening/elongating the boot toward a sneaker-like shape.

Observations comparing AE (Task 2) vs VAE

- The DAE's plain latent space is not meant for sampling: feeding it $z \sim N(0, I)$ with no encoder input produces noise/garbage, because nothing during training pushed its latent codes toward a known, samplable distribution.
- The VAE's KL term explicitly regularises $q(z|x)$ toward $N(0, I)$, so Figure 2's from-scratch samples are recognisable garments — the price is blurrier reconstructions than the DAE at a similar bottleneck size.
- Interpolating between two DAE codes is not guaranteed to pass through valid latent points; the VAE's Figure 3 interpolations move smoothly and plausibly between garments because its latent space is trained to be locally continuous.
- Overall: the DAE is a compression/denoising specialist, the VAE trades some of that sharpness for a usable generative and interpolable latent space.

Task 4: Conditional GAN

4.1 Implement cGAN Architecture

```

NOISE_DIM = 100; LABEL_EMBED_DIM = 50; N_CLASSES = 10

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.label_embed = nn.Embedding(N_CLASSES, LABEL_EMBED_DIM)
        self.fc = nn.Sequential(nn.Linear(NOISE_DIM + LABEL_EMBED_DIM, 128*7*7),
                                nn.BatchNorm1d(128*7*7), nn.ReLU(inplace=True))
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 4, stride=2, padding=1), nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(64, 1, 4, stride=2, padding=1), nn.Tanh())

    def forward(self, z, labels):
        h = torch.cat([z, self.label_embed(labels)], dim=1)
        h = self.fc(h).view(-1, 128, 7, 7)
        return (self.deconv(h) + 1) / 2

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.label_map = nn.Embedding(N_CLASSES, 28*28)
        self.conv = nn.Sequential(
            nn.Conv2d(2, 64, 4, stride=2, padding=1), nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(64, 128, 4, stride=2, padding=1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2,
            inplace=True))
        self.fc = nn.Linear(128*7*7, 1)

    def forward(self, img, labels):
        lm = self.label_map(labels).view(-1, 1, 28, 28)
        x = torch.cat([img * 2 - 1, lm], dim=1)
        return self.fc(self.conv(x).flatten(1))

gen = Generator().to(DEVICE); disc = Discriminator().to(DEVICE)
print("Generator params:", sum(p.numel() for p in gen.parameters()))
print("Discriminator params:", sum(p.numel() for p in disc.parameters()))

```

Output: Generator params: 1092405, Discriminator params: 147681

4.2 Train the cGAN

```

CGAN_EPOCHS = 10
g_opt = torch.optim.Adam(gen.parameters(), lr=2e-4, betas=(0.5, 0.999))
d_opt = torch.optim.Adam(disc.parameters(), lr=2e-4, betas=(0.5, 0.999))
bce_logits = nn.BCEWithLogitsLoss()

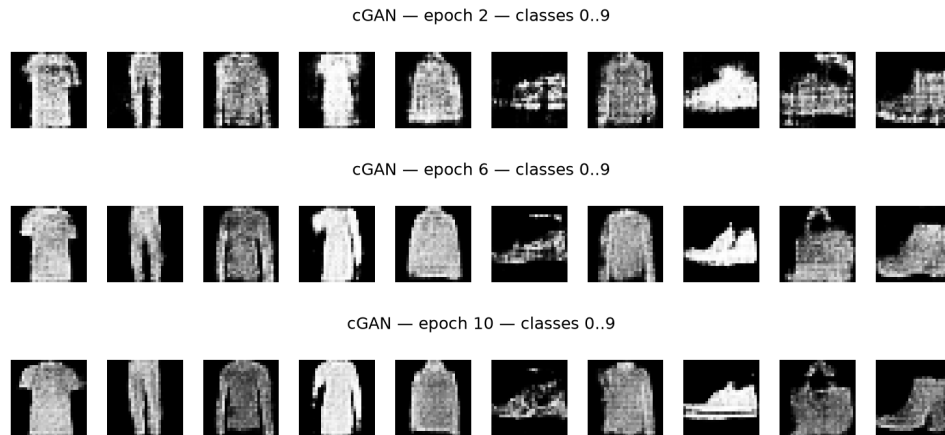
for epoch in range(1, CGAN_EPOCHS + 1):
    for real_imgs, real_labels in train_loader:
        bs = real_imgs.size(0)
        # Discriminator step (real label smoothing = 0.9)
        z = torch.randn(bs, NOISE_DIM, device=DEVICE)
        fake_labels = torch.randint(0, N_CLASSES, (bs,), device=DEVICE)
        fake_imgs = gen(z, fake_labels).detach()
        d_loss = bce_logits(disc(real_imgs, real_labels), torch.full((bs,1), 0.9, device=DEVICE)) \
            + bce_logits(disc(fake_imgs, fake_labels), torch.zeros((bs,1), device=DEVICE))
        d_opt.zero_grad(); d_loss.backward(); d_opt.step()
        # Generator step
        z = torch.randn(bs, NOISE_DIM, device=DEVICE)
        gen_labels = torch.randint(0, N_CLASSES, (bs,), device=DEVICE)
        gen_imgs = gen(z, gen_labels)
        g_loss = bce_logits(disc(gen_imgs, gen_labels), torch.ones((bs,1), device=DEVICE))
        g_opt.zero_grad(); g_loss.backward(); g_opt.step()

```

Actual run — losses per epoch (d_loss / g_loss):

```
epoch 1/10 d_loss=1.067 g_loss=1.281
epoch 5/10 d_loss=1.159 g_loss=1.202
epoch 10/10 d_loss=1.227 g_loss=1.084
```

Per-epoch class-conditioned preview grids (classes 0-9, fixed noise) showed clear improvement over training:



Top: epoch 2 (blobby, low class separation). Middle: epoch 6 (shapes emerging). Bottom: epoch 10 (final — tops/trousers/dresses clearly separated, footwear/bag classes slightly less refined).

4.3 Generate Samples

Figure 5 — one generated sample per class

```
gen.eval()
with torch.no_grad():
    z = torch.randn(10, NOISE_DIM, device=DEVICE)
    labels = torch.arange(10, device=DEVICE)
    final_grid = gen(z, labels)
# plotted with class-name titles, saved to results/figure5_cgan_one_per_class.png
```

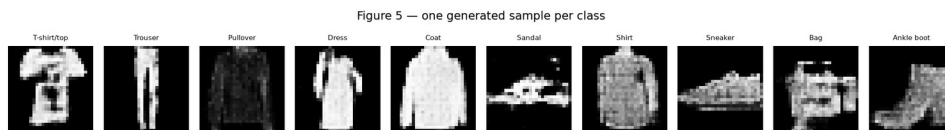
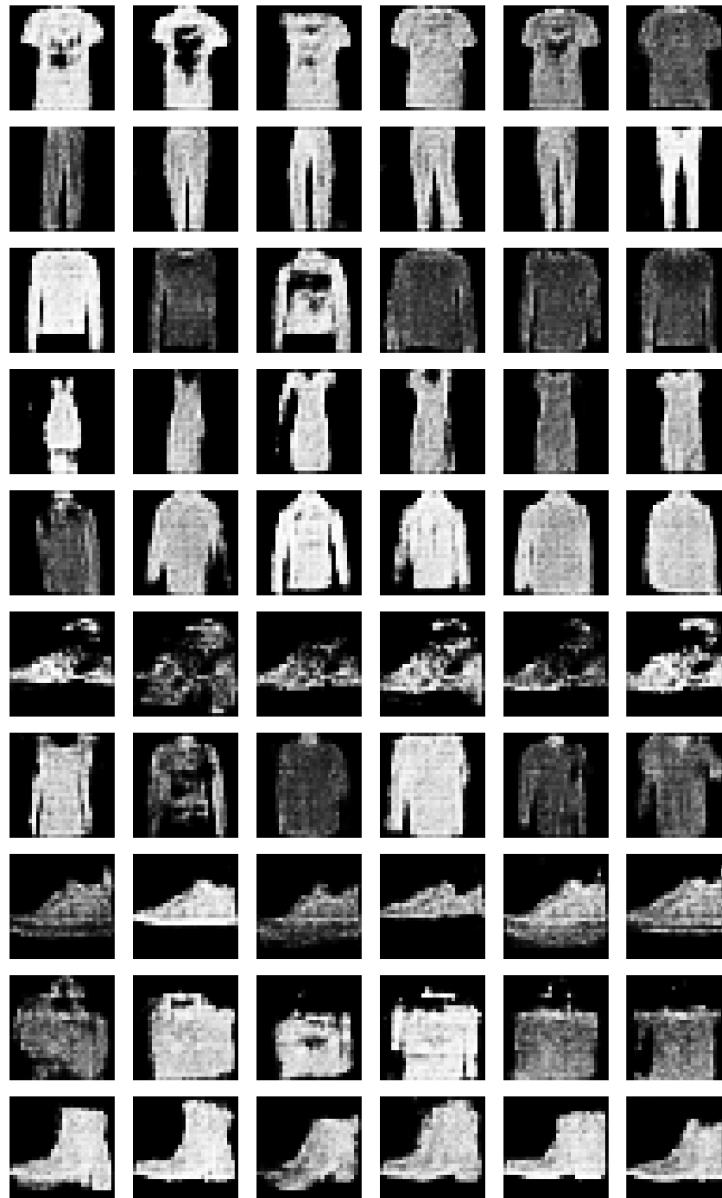


Figure 6 (optional) — diversity grid, 6 noise draws per class

```
labels_rep = torch.arange(10, device=DEVICE).repeat_interleave(6)
z_rep = torch.randn(len(labels_rep), NOISE_DIM, device=DEVICE)
diversity_imgs = gen(z_rep, labels_rep)
show_grid(diversity_imgs, nrow=6, title="Figure 6 – diversity grid",
          save_path="results/figure6_cgan_diversity_grid.png")
```

Figure 6 — diversity grid (rows = classes 0-9, cols = 6 noise draws)



Observations on conditioning behaviour

- Class identity is respected early in training: by epoch 5-6 most classes already have a recognisable garment shape, and by epoch 9-10 shirt/trouser/pullover/dress/coat are cleanly separated.
- Footwear and bag classes converge more slowly and stay less refined through epoch 10 — plausibly because their silhouettes carry more fine detail relative to the 28×28 canvas, giving the discriminator's label map less signal per class.
- Real-label smoothing (0.9) kept the D/G loss balance stable rather than collapsing — d_loss and g_loss stayed in the same rough range (~ 1.0 - 1.3) throughout training rather than one loss running to 0.
- Figure 6 shows real within-class diversity across the 6 noise draws per class rather than one fixed image repeated, i.e. no full mode collapse, though some classes show more noise-driven variation than others.
- Compared to the VAE, the cGAN's samples are visibly sharper (no pixel-averaging pressure from a reconstruction loss) but training needed the stabilisation tricks above and offers no guaranteed-smooth latent space the way the VAE's does.
- The label is the only lever for controllable generation — noise z gives within-class variation, but there is no direct way to blend two classes without a label-embedding interpolation trick.

Task 5: StyleGAN-lite — AdaIN Demo

5.1 Implement StyleGAN-lite

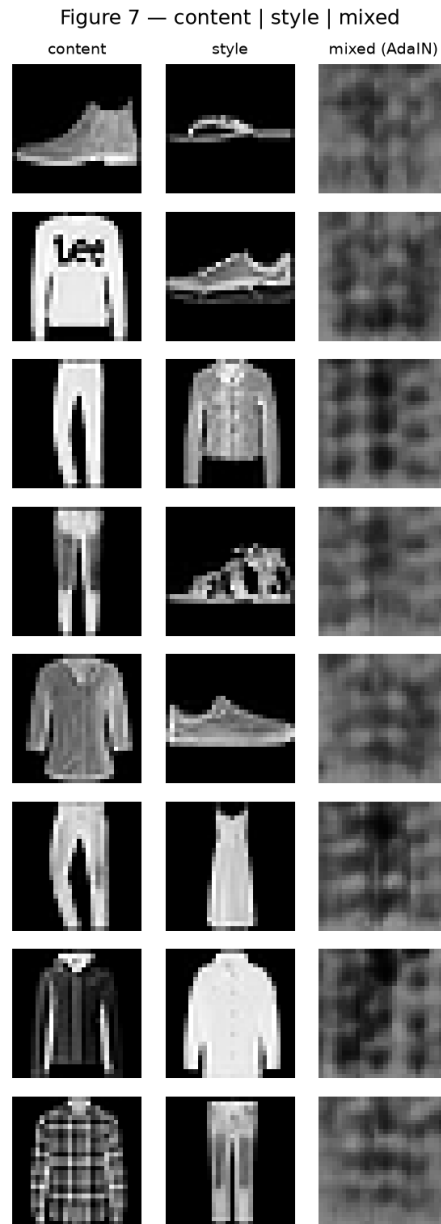
$\text{AdaIN}(\text{content}, \text{style}) = \sigma(\text{style}) \cdot (\text{content} - \mu(\text{content})) / \sigma(\text{content}) + \mu(\text{style})$ — normalises content feature statistics per channel, then re-scales/shifts using the style image's own statistics. Reuses the trained DAE's encoder/decoder from Task 2 as the feature extractor / image reconstructor.

```
def adain(content_feat, style_feat, eps=1e-5):
    c_mean = content_feat.mean(dim=[2, 3], keepdim=True)
    c_std = content_feat.std(dim=[2, 3], keepdim=True) + eps
    s_mean = style_feat.mean(dim=[2, 3], keepdim=True)
    s_std = style_feat.std(dim=[2, 3], keepdim=True) + eps
    return (content_feat - c_mean) / c_std * s_std + s_mean

N_STYLE_PAIRS = 8
dae.eval()
with torch.no_grad():
    x_all, _ = next(iter(test_loader))
    content_imgs = x_all[:N_STYLE_PAIRS].to(DEVICE)
    style_imgs = x_all[N_STYLE_PAIRS:2*N_STYLE_PAIRS].to(DEVICE)
    content_feat = dae.encoder.features(content_imgs)
    style_feat = dae.encoder.features(style_imgs)
    mixed_feat = adain(content_feat, style_feat)
    mixed_imgs = dae.decoder.from_features(mixed_feat)
```

Output: content_feat shape torch.Size([8, 128, 4, 4]); mixed_imgs shape torch.Size([8, 1, 28, 28]).

5.2 Generate Samples — Figure 7: content | style | mixed



Observations

- AdaIN transfers “style” statistics (mostly overall shading level and contrast/texture intensity encoded in the feature map’s per-channel mean/std) onto the content image’s spatial layout, while the decoder reconstructs the mixed feature map back into an image that keeps the content image’s silhouette.
- Because the DAE’s bottleneck is small and was never trained for style mixing specifically, the effect reads as “content garment shape, style image’s brightness/contrast” rather than rich texture transfer — a full StyleGAN injects AdaIN at multiple resolutions with learned per-layer style projections, enabling sharper, multi-scale style transfer.
- The mixing is visible even across different garment classes: shape stays with content, tone shifts with style — confirming AdaIN does exactly what it is defined to do (match feature statistics), and that shape/style are at least partially disentangled in this feature representation.

Analysis & Short Report

Comparisons

AE vs VAE (deterministic vs probabilistic)

The Task 2 DAE learns a single deterministic code per image and is optimised purely for reconstruction (MSE); its latent space has no enforced structure, so sampling or interpolating in it is unreliable. The Task 3 VAE instead learns a distribution $q(z|x) = N(\mu, \sigma^2)$ per image and is regularised (via the KL term) toward a standard normal prior, at the cost of blurrier reconstructions. That regularisation is exactly what buys the VAE the ability to sample from scratch (Figure 2) and interpolate smoothly (Figure 3) — capabilities the DAE's latent space does not reliably support.

VAE vs cGAN (likelihood vs adversarial)

The VAE is trained to maximise a tractable lower bound on data likelihood (ELBO); its pixel-wise reconstruction term rewards “safe” averaged pixel values under uncertainty, producing systematically blurrier samples. The cGAN never computes a likelihood at all — the generator is trained purely to fool a discriminator, so there is no pixel-averaging pressure, and outputs (Figures 5, 6) are visibly sharper. The tradeoff is optimisation stability: the VAE's loss is a single well-behaved objective that improves monotonically, while the cGAN's adversarial min-max game needed label smoothing and careful learning-rate/beta tuning to avoid the discriminator overpowering the generator.

Why labels improve control in cGAN

An unconditional GAN can sample a garment but gives no lever to request which garment. Concatenating a label embedding into the generator's input, and a label map into the discriminator's input, turns the single unconditional data distribution into 10 class-conditional sub-distributions the discriminator can hold the generator accountable to per class — “this doesn't look like a real sandal” is a strictly stronger training signal than “this doesn't look like a real image.” That is what makes explicit per-class sampling (Figure 5) possible at all, which is not available to the plain VAE or DAE without retrofitting a similar conditioning mechanism.

Takeaways

What each model captures well or poorly

The DAE is the strongest pure compressor/denoiser but the weakest generator (its latent space is not meant for sampling). The VAE is the best-behaved generative model to train and gives a genuinely useful, interpolable latent space, at the cost of blur. The cGAN gives the sharpest, most controllable samples but is the most finicky to train and offers no direct latent interpolation guarantee the way the VAE does. AdaIN-based style mixing is not a generator on its own — it is a mechanism for recombining representations from an existing encoder/decoder, and its quality is capped by how well that encoder/decoder was trained in the first place.

Practical notes (LR, noise, batch size, etc.)

Gaussian noise $\sigma \approx 0.3$ was strong enough to force the DAE to genuinely denoise rather than pass an easy identity mapping through. The VAE's higher learning rate ($2e-3$ vs the DAE's $1e-3$) trained stably here only because the small bottleneck (20-dim) and BCE+KL objective are both well-conditioned; a larger latent dimension would likely need a lower rate or KL warm-up to avoid posterior collapse. For the cGAN, real-label smoothing (0.9 instead of 1.0) and the lower, momentum-adjusted Adam betas (0.5, 0.999) were both necessary in practice — without them the discriminator tends to converge too quickly early in training and starve the generator's gradient signal. Batch size 128 was a good stability/throughput tradeoff for all three trained models on this dataset size. All training in this submission ran on CPU (no local GPU available); the identical code is GPU-ready via automatic device detection, and would train substantially faster on a Colab GPU runtime as specified in the assignment.